arXiv:2610.33716v1 [cs.CR] 24 Oct 2026

VulDec: Vulnerability Detection in Binaries via Neural Decompilation

Francesco Crescenzo Grasso[1*], Pietro Liguori[1], Roberto Natella[2], Domenico Cotroneo[3]

[1*]DIETI, Università degli Studi di Napoli Federico II, Via Claudio 21, Naples, 80125, Campania, Italy.
[2]Computer Science, Gran Sasso Science Institute, Viale Francesco Crispi 7, L'Aquila, 67100, Abruzzo, Italy.
[3]Department of Computer Science, University of North Carolina at Charlotte, 9201 University City Blvd, Charlotte, 28223, North Carolina, USA.

*Corresponding author(s). E-mail(s): francescocrescenzo.grasso@unina.it;
Contributing authors: pietro.liguori@unina.it; roberto.natella@gssi.it; d.cotroneo@charlotte.edu;

Abstract

Automated vulnerability detection in binary executables is challenging due to the lack of source code and the limited semantic information available at the assembly level. Despite significant progress in machine-learning-based code analysis, the field still lacks methodologies and large, high-quality datasets that provide vulnerable code jointly at assembly- and source-level with precise vulnerability labels. To address this gap, we introduce VulDec, consisting of (i) a new large-scale, carefully curated dataset aligning real-world Assembly–C function pairs with detailed CWE annotations across multiple architectures, and (ii) the first end-to-end framework that leverages neural decompilation for binary vulnerability detection. Tuned LLM-based neural decompilers trained on our dataset achieve strong source-level recovery across architectures (Edit Similarity of approximately 56%, BLEU-4 of approximately 43%), while our tuned LLM-based detectors reach a macro F1-score of 0.61 on an in-distribution realistic test set (3% vulnerability rate). Our findings provide evidence that neural decompilation offers a powerful alternative to conventional low-level binary representations, facilitating robust binary vulnerability detection.

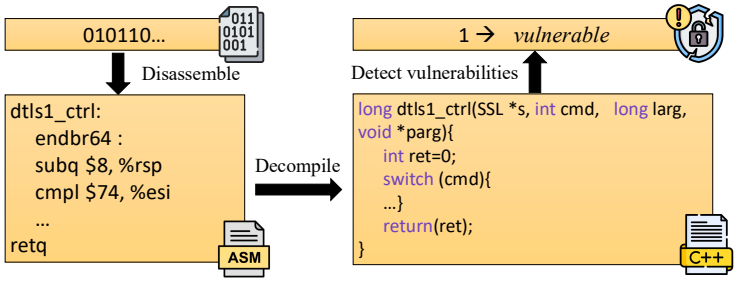


Fig. 1: Illustrative example of the VulDec pipeline applied to a binary from *OpenSSL*

Keywords: Automated Software Security, Generative Software Engineering, Neural Decompilation, Binary Vulnerability Detection, Reverse Engineering

1 Introduction

Binary-based vulnerability detection represents a critical challenge in automated and trustworthy software engineering, as vulnerabilities in executable files can be exploited to compromise entire systems. Unlike approaches that rely on source code (Chakraborty et al. 2022; Li et al. 2018; Russell et al. 2018; Zhou et al. 2019), binary-based vulnerability detection is essential in scenarios where the source code is unavailable, such as proprietary software, legacy systems, and firmware embedded in specialized hardware. As a matter of fact, many attack techniques target compiled binaries rather than source code, making binary-based analysis useful for robust system security (Shoshitaishvili et al. 2016; Wu et al. 2025).

Several studies have explored binary-based vulnerability detection by leveraging different low-level representations, such as disassembled code (Dahl et al. 2020), LLVM IR (Schaad and Binder 2023; Zheng et al. 2019), and code graphs (Eschweiler et al. 2016; Grieco et al. 2016; Pewny et al. 2015). However, these representations largely preserve assembly-level structure and semantics, limiting their suitability for vulnerability detection approaches that rely on high-level abstractions rather than low-level instruction patterns.

Since vulnerability detection in source code is far more mature and supported by a well-established state of the art, binary-level detection can be approached as a standard vulnerability detection task by using *decompilation*, which converts binaries into high-level code and allows existing techniques to be applied.

As found by Cao et al. (2024), traditional decompilers struggle to generate human-readable and well-structured code, often producing output that remains too close to raw assembly. Consequently, they fail to fully exploit the advantages of high-level program abstraction, reducing their effectiveness in automated security analysis.

Therefore, we propose the use of *neural decompilation*, a generative approach that leverages Large Language Models (LLMs) to translate binaries back into source

code. The process begins by disassembling the binary into assembly code, followed by decompilation into high-level code. This methodology produces a decompiled program that closely mirrors the original source, utilizing the transformer encoder-decoder architecture to capture the semantic richness inherent in real-world code.

Gaining access to the reconstructed source code supports a range of further applications, from malware identification to comprehensive static analysis, and, in particular, enables automated vulnerability detection.

Along this direction, vulnerability detection on decompiled code poses non-trivial challenges. Neurally-decompiled programs, as pointed out by Hosseini and Dolan-Gavitt (2022), are often incomplete, non-canonical, and lack the semantic guaranties expected by source-level analysis tools, such as precise type information and whole-program context. Most traditional methods do not natively operate at function-level granularity (Pistoia et al. 2007); moreover, they tend to produce a high number of false positives (Ayewah et al. 2007). As a result, static vulnerability detectors tend to degrade or fail when applied to decompiled code. In this context, machine learning-based approaches, and in particular LLMs, are an appealing solution to mitigate the approximations and uncertainty introduced by decompilation, making them well-suited for vulnerability detection.

A further issue concerns existing state-of-the-art datasets: they address neural decompilation and vulnerability detection in isolation. Decompilation benchmarks (da Silva et al. 2021; Hosseini and Dolan-Gavitt 2022; Katz et al. 2018; Yang et al. 2011) provide source–binary pairs without security annotations, while vulnerability datasets (Chakraborty et al. 2022; Chen et al. 2023; Fan et al. 2020; Ni et al. 2024; NIST 2017; Zhou et al. 2019) are mostly at the source-level and lack corresponding binaries. Although there exist CWE-annotated datasets that can be compiled to obtain corresponding binaries, e.g., the Juliet Test Suite C/C++ (NIST 2017), these resources typically rely on synthetic code. Prior work (Chakraborty et al. 2022) has shown that synthetic samples often fail to capture real-world vulnerability patterns, thereby limiting their effectiveness in practical scenarios. Since detection operates on pseudo-C code generated by the decompiler, training both models on a shared data distribution ensures semantic consistency across pipeline stages and mitigates distribution shift between training and inference.

In light of these concerns, we propose VulDec, an end-to-end framework to detect vulnerabilities in binaries using *neural decompilation* and *LLM-based vulnerability detection*. Figure 1 shows an illustrative example of our pipeline, from binary to vulnerability detection. To enable end-to-end approaches that bridge different abstraction levels, our framework includes a large-scale, carefully curated dataset that provides a direct mapping between real-world C–Assembly function pairs, complemented by detailed CWE annotations across multiple architectures.

This work aims to demonstrate that neural decompilation constitutes an effective approach for binary vulnerability detection, supporting its adoption for this purpose.

Our contributions are as follows:

- We introduce the VulDec framework, an automated LLM-based decompilation-classification pipeline to detect vulnerabilities in binaries.

- Given the lack of large-scale datasets explicitly linking low-level assembly to high-level vulnerable source code, we present the VulDec dataset [1], a collection of more than 59,000 (non-)vulnerable C-ASM program pairs extracted from 7,000 real-world commits, whose assembly is lifted directly from official Debian packages (The Debian Project 2025).
- We perform an extensive evaluation, showing that fine-tuned LLM-based neural decompilers achieve strong source-level recovery across architectures (Edit Similarity $\approx$ 56%, BLEU-4 $\approx$ 43%), while our tuned LLM-based detectors reach a macro F1-score of 0.61 on an in-distribution realistic test set (3% vulnerability rate).

The remainder of the paper is structured as follows. In Section 2, we elaborate on background concepts related to the target problem. In Section 3, we describe the construction of the dataset and the methodology adopted for vulnerability prediction through neural decompilation. In Sections 4 and 5, we illustrate the experimental evaluation and analysis for the decompilation and vulnerability prediction tasks. In Section 6, we analyze and interpret the results of the model evaluation. In Section 7, we examine related work. In Section 8, we discuss the threats to validity. Section 9 concludes the paper.

2 Background

Decompilation aims to recover high-level program representations from executable code, reversing the transformations applied during compilation and linking. While the forward compilation pipeline, i.e., from high-level language to assembly, object code, and finally executable, is well defined, the reverse process is inherently ill-posed. Disassembly, which lifts binary code to assembly, is largely supported by mature tools; however, no deterministic inverse exists for decompilation, as compilation irreversibly discards high-level semantic information. This challenge primarily arises from the *semantic gap* between high- and low-level code. Executable binaries lack explicit abstractions such as functions, data types, and structured control constructs, which are instead implemented through low-level jumps, calls, and register manipulations. As a result, reconstructing the original control flow and program structure from binary code is highly non-trivial. An additional drawback is the strong *architectural dependence* (e.g., x86, ARM, MIPS) of low-level code. While high-level languages are largely architecture-agnostic, executable binaries are tightly coupled to the target instruction set, calling conventions, and register models. This dependency further complicates decompilation, as the same high-level construct may be realized through substantially different instruction sequences across architectures.

Traditional decompilers rely on static program analysis techniques to reconstruct high-level representations from binary executables. Widely used tools such as Ghidra (National Security Agency 2019), IDA Pro (Hex-Rays 2025), and RetDec (Avast Software 2017) operate on intermediate representations; P-code in Ghidra, microcode IR in IDA, and LLVM IR in RetDec. They are derived from control-flow graphs (CFGs), where basic blocks and their transitions are explicitly modeled, and may further enrich

[1] Available on Figshare.

them with data-flow graphs (DFGs) and program dependence graphs (PDGs) to track variable usage and dependencies.

However, as stated by Cao et al. (2024) and Cotroneo et al. (2025), traditional decompilers exhibit intrinsic limitations rooted in their reliance on static analysis and rule-based transformations. Their effectiveness critically depends on the availability of well-formed, *fully compilable binaries*, as accurate control-flow and data-flow recovery require complete functions and closed control-flow graphs. This constraint limits their applicability in scenarios involving partial code fragments, dynamically generated code, or isolated binary snippets. Furthermore, despite traditional decompilers producing syntactically valid outputs, the reconstructed representations are often semantically approximate, *assembly-like*, and verbose, reflecting low-level implementation details rather than high-level program intent. Intermediate representations are designed for deterministic structural analysis rather than semantic abstraction. Finally, the strong coupling to predefined architectures and handcrafted heuristics hinders robustness across different compiler configurations and target platforms, motivating the exploration of neural decompilation approaches that relax these assumptions by learning semantic patterns directly from data.

Previously explored by several works (Cotroneo et al. 2025; Hosseini and Dolan-Gavitt 2022; Katz et al. 2018; Tan et al. 2024), *neural decompilation* has emerged as a promising alternative to traditional rule-based approaches by framing decompilation as a learning problem rather than a deterministic reconstruction task. Instead of explicitly modeling control-flow recovery and semantic lifting through handcrafted heuristics, neural models learn latent representations that capture recurring semantic patterns across large corpora of compiled programs. This paradigm enables training on paired sequences of low-level and high-level code, effectively casting decompilation as a *translation task*. Such a data-driven approach facilitates the implicit modeling of semantic patterns that are difficult to encode explicitly, allowing neural decompilers to recover representations that are generally more *C-like*, structured, and readable than those produced by traditional tools. A key advantage lies in their increased *flexibility* with respect to input completeness and variability, as neural models can operate on code fragments or isolated functions, making them applicable even when fully compilable binaries are unavailable.

Despite these advantages, neural decompilation also presents notable challenges. Code generation models inherently carry the risk of not consistently providing syntactically correct or relevant code. The performance of neural models heavily depends on the availability and quality of large-scale paired datasets of binaries and source code, whose construction is non-trivial and may introduce bias toward specific compilers, optimization levels, or programming styles. Additionally, neural decompilers may struggle with generalization beyond the training distribution and often lack the interpretability and deterministic guarantees provided by traditional analysis-based tools. As a result, neural decompilation should be viewed not as a replacement but as a complementary approach to traditional decompilation, particularly in contexts requiring scalable and semantics-aware analysis.

Vulnerability detection is a well-established problem in software security, aiming to identify programming errors and security flaws that may lead to exploitable behaviors.

Existing approaches can be broadly categorized into source-level and binary-level vulnerability detection. Source-level techniques (Chakraborty et al. 2022; Chen et al. 2023; Cotroneo et al. 2025; Li et al. 2018; Russell et al. 2018; Wheeler 2021; Zhou et al. 2019) operate on high-level code representations and benefit from explicit program semantics, structured control flow, and rich contextual information, enabling precise analysis through static analysis, dynamic analysis, symbolic execution, or learning-based methods. ML-based approaches (Dahl et al. 2020; Eschweiler et al. 2016; Grieco et al. 2016; Schaad and Binder 2023; Pewny et al. 2015; Zheng et al. 2019) are particularly suitable for large or diverse code fragments, as they can capture recurring vulnerability patterns without requiring extensive handcrafted rules. In contrast, binary-level vulnerability detection operates directly on executable code, where the lack of high-level abstractions and semantic information significantly complicates the analysis.

Decompilation plays a crucial role in enabling *binary-level vulnerability detection* by recovering higher-level program representations from executables, thus allowing the application of mature source-level vulnerability detection techniques in binary-only settings. In this context, neural decompilation, by producing more structured and semantically expressive representations, provides a promising mechanism for transferring source-level security analyzes to binaries. However, despite this potential, decompilation has largely remained a support tool for code readability and manual reverse engineering (Hosseini and Dolan-Gavitt 2022), rather than being systematically integrated into automated security analysis pipelines. Consequently, existing approaches often fail to fully exploit recovered semantic information for security-driven tasks, motivating the development of methods that tightly couple neural decompilation with vulnerability detection to enable scalable and effective binary-level analysis.

Existing datasets typically support neural decompilation and vulnerability detection in isolation. Decompilation datasets (da Silva et al. 2021; Hosseini and Dolan-Gavitt 2022; Katz et al. 2018; Yang et al. 2011) focus on paired source–binary representations but lack security annotations, whereas vulnerability detection datasets (Chakraborty et al. 2022; Chen et al. 2023; Fan et al. 2020; Ni et al. 2024; NIST 2017; Russell et al. 2018; Zhou et al. 2019) are primarily defined at the source level and do not include corresponding binaries. Many widely used vulnerability datasets are synthetic (NIST 2017), cover a limited set of vulnerability patterns (Russell et al. 2018), or do not strictly follow standardized taxonomies such as CWE (Chakraborty et al. 2022; Zhou et al. 2019). Moreover, they rarely ensure full compilability (Chakraborty et al. 2022; Chen et al. 2023; Fan et al. 2020; Zhou et al. 2019), limiting their applicability in studying how source-level vulnerabilities manifest at the binary level. As a consequence, unified datasets that jointly capture source code, binaries, and vulnerability annotations remain largely unavailable, hindering the development of end-to-end approaches that integrate decompilation and vulnerability detection.

3 Methodology

This section introduces the proposed VulDec framework for binary vulnerability detection and the construction of the VulDec dataset used to support its training and evaluation.

3.1 Dataset Construction

The effectiveness of neural decompilation for vulnerability detection critically depends on the availability of a high-quality dataset that provides aligned source-level and binary-level representations, along with reliable vulnerability annotations. To this end, we constructed VulDec, a corpus of C–Assembly function pairs enriched with standardized vulnerability labels, with a particular emphasis on reproducibility and coverage of real-world software. Figure 2 shows the steps leading to the construction of the dataset.

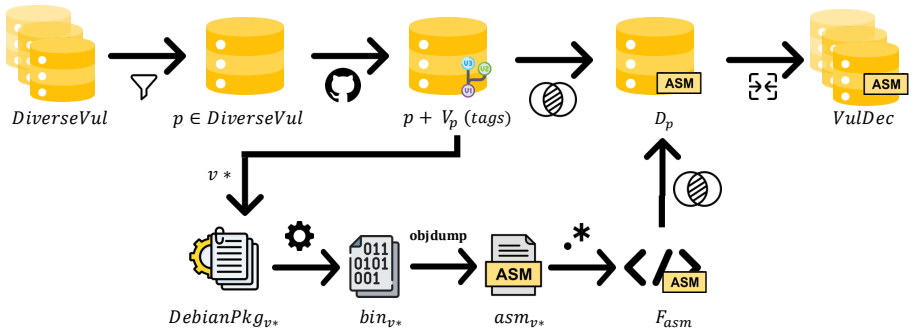


Fig. 2: Automated compilation, cross-architecture assembly generation and source-level alignment

We build upon the *DiverseVul* dataset by Chen et al. (2023), a large-scale collection of vulnerable and non-vulnerable C/C++ functions mined from real-world open-source projects and annotated via vulnerability-fixing commits. Although DiverseVul does not provide directly compilable source projects, it offers a uniquely large and diverse set of source-level functions with reliable, standardized vulnerability annotations, overcoming the limitations related to the synthetic datasets (Chakraborty et al. 2022) and those reported by Jain et al. (2023). With approximately 19,000 vulnerable and over 330,000 non-vulnerable functions spanning 150 CWE categories, DiverseVul provides the scale required to support data-intensive neural models, motivating its selection as the foundation of our dataset. To obtain the binary representations required for neural decompilation, we leverage the availability of *Debian source packages* (The Debian Project 2025), which provide curated, reproducible, and compilable versions of many widely used open-source projects.

The process, formalized in the Algorithm 1, begins with the systematic selection of software projects from the DiverseVul. For each project p, a local copy is instantiated via git clone to enable deep temporal analysis of the codebase. The alignment phase leverages the project's versioning history. We extract the set of all available release tags (V_p) to map the specific vulnerability samples (S_p), comprising the source function f_c, the ground truth label t, the commit identifier c_{id}, and the CWE category, onto

Algorithm 1 Dataset Construction from DiverseVul

$VulDec \leftarrow \emptyset$
for all $p \in DiverseVul$ **do**
  `git clone`(p_{url})
  $V_p \leftarrow$ `git tag`(p)
  $S_p \leftarrow \{s = (f_c, t, c_{id}, \text{cwe}) \mid s \in p\}$
  $A_p \leftarrow \{v \in V_p \mid \exists\ \text{DebianPkg}_v\}$
  if $A_p \neq \emptyset$ **then**
    $v^* \leftarrow \arg\max_{v \in A_p} |S_p(v)|$
    $bin_{v^*} \leftarrow \text{build}(\text{DebianPkg}_{v^*})$
    $asm_{v^*} \leftarrow \text{disassemble}(bin_{v^*})$
    $f_{asm} \leftarrow \text{f_extract}(asm_{v^*})$
    for all $s \in S_p$ **do**
      if $\text{name}(f_{asm}) = \text{name}(s.f_c)$ **then**
        $D_p \leftarrow D_p \cup \{(s.f_c,\ f_{asm},\ s.t,\ s.c_{id},\ s.cwe)\}$
      end if
    end for
    $VulDec \leftarrow VulDec \cup D_p$
  end if
end for
return $VulDec$

stable software versions. To ensure the reproducibility and reliability of the compilation environment, we define A_p as the subset of versions for which a corresponding Debian package is available. A critical step in our methodology is the selection of the optimal version v^*. This is achieved through a greedy maximization strategy:

$$v^* \leftarrow \text{argmax}_{v \in A_p} |S_p(v)|,$$

where $|S_p(v)|$ is the number of samples obtained from version v. This choice is made to ensure that we capture the highest possible density of vulnerability samples for each project if, and only if, $A_p \neq \emptyset$, i.e, at least one version of the project is supported by the Debian build system; otherwise, we skip to the next project. Once the optimal package DebianPkg_{v^*} is identified, it is compiled into a binary executable (bin_{v^*}). We then use a disassembler utility to obtain the assembly representation (asm_{v^*}) of the binary code, as well as the names (symbols) and addresses of functions in the code. From this output, the function boundaries are identified and extracted (f_{asm}). The dataset for a project (D_p) is then populated through a name-based intersection: a sample is included only if there is an equality between the symbol name in the assembly f_{asm} and the function name in the source code f_c.

In rare instances involving homonymous functions, a manual verification was conducted by the authors to resolve potential mapping ambiguities. We took measures to mitigate any systematic errors arising from such edge cases, ensuring their impact is negligible; a detailed discussion on these factors is provided in Section 8.

In order to reduce syntactic variability and facilitate downstream learning tasks, both C and assembly representations undergo a normalization phase aimed at abstracting literal strings using a placeholder token STR, removing non-semantic artifacts (// and $/*...*/$ comments, pre-processor and call frame information directives, irregular spacing) while preserving the functional structure of the code.

This rigorous automated process yields the global dataset, a collection of high-fidelity, cross-domain pairs ready for downstream tasks.

3.2 Proposed Framework

This work introduces VulDec, an end-to-end framework for vulnerability detection directly from binary executables. Given a compiled binary, VulDec first extracts its assembly representation using a static, architecture-aware disassembler. The resulting assembly code is then processed by a neural decompilation module to reconstruct a high-level approximation of the original source code. Finally, a neural vulnerability detection module analyzes the recovered source representation to determine whether the function contains security vulnerabilities. Figure 3 illustrates the overall pipeline and the intermediate representations produced at each stage.

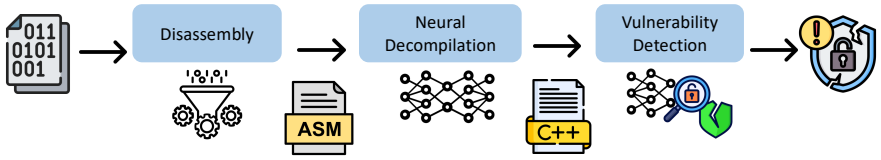


Fig. 3: VulDec workflow: binary-level function analysis and vulnerability discovery

As shown in Figure 4, the framework is trained through two distinct yet complementary learning tasks. Neural decompilation is formulated as a *sequence-to-sequence* translation task, in which pre-trained large language models are fine-tuned to map sequences of assembly instructions to semantically equivalent high-level source code written in C or C++. The objective is to learn a conditional transformation from low-level binary representations to structured, human-readable code. All samples are defined at the function level, with variable-length instruction and token sequences, and the decompilation process operates in a controlled setting that excludes compiler optimizations and code obfuscation.

Vulnerability detection, in contrast, is formulated as a binary classification task. Here, pretrained large language models are fine-tuned to analyze the decompiled C/C++ code and predict whether a function is vulnerable (1) or non-vulnerable (0). The detection model is trained on C/C++ source code and is applied at inference to decompiled pseudo-C/C++, reflecting realistic deployment conditions. By integrating disassembly, neural decompilation, and automated vulnerability analysis, VulDec enables vulnerability assessment in the absence of the original source code.

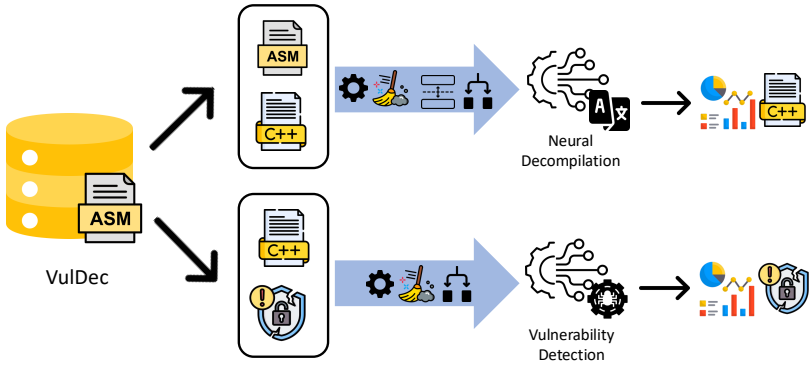


Fig. 4: VulDec training and testing workflow: neural decompilation and vulnerability detection. Note: integrated data sources (Section 4) are used to enhance detection diversity

4 Experimental Setup

All experiments were conducted on a workstation equipped with an *NVIDIA GTX* 1660 GPU with 6 GB of VRAM and 32 GB of system RAM. For models exceeding 1 billion parameters, an *NVIDIA T4* GPU with 16 GB of VRAM was additionally utilized, along with *Low-Rank Adaptation* (LoRA) to mitigate the computational load. This hardware setup, combined with memory-efficient adaptation techniques, proved sufficient to support training and inference across all experimental phases, enabling repeated evaluation cycles and consistent comparisons of model configurations and preprocessing choices.

4.1 Dataset Overview

VulDec is a real-world C/ASM dataset for function-level vulnerability analysis across x86 and ARM architectures. We adopted two widely deployed ISAs (Instruction Set Architectures), representative of both server and embedded/IoT domains. These architectures follow fundamentally different design philosophies (CISC and RISC), allowing us to capture substantial variability at the assembly level and evaluate the generality of the proposed approach across clearly different low-level representations. The automated compilation of the binary artifacts was performed using the standard Debian toolchain (via `apt-get --build source`). We targeted the source packages through the distribution's build utilities, employing the GNU Compiler Collection (GCC) version 12.2.0 compiler suite with the `-O2` optimization level to ensure a consistent and reproducible production environment. The assembly code is obtained from binary executables using `llvm-objdump` (The LLVM Foundation 2026), a static, architecture-aware disassembler operating at the instruction level.

Table 1 shows some relevant statistics of VulDec. The ARM and x86 subsets share 23,163 common functions, representing functions present in both architectures. The dataset contains no duplicate entries or missing values, ensuring data consistency.

Table 1: VulDec statistics on projects, CWEs, functions, and code metrics

Dataset	ARM	x86	Dataset	ARM	x86
# *functions*	28,426	31,340	# *projects*	24	24
# *vulnerable func.*	1,879	1,949	# *CWE Types*	80	74
% *vulnerable rate*	6.6%	6.2%	*LOC (median)* C/C++	18	18
# *common func.*	23,163	-	*LOC (median)* ASM	56	49

Vulnerability prevalence is low and reflects real-world conditions. Both architecture-specific subsets exhibit a vulnerability rate of approximately 6–7%. This is slightly higher than the original DiverseVul rate of 5.7% (Chen et al. 2023), leading to a marginally more balanced yet still realistic class distribution.

VulDec covers a diverse set of widely used open-source projects, including Linux, OpenSSL, QEMU, FFmpeg, and Git, with substantial variability in project size. Across both architectures, the most prevalent vulnerability classes are consistently CWE-703, CWE-119, and CWE-20, indicating that improper control flow validation, memory corruption vulnerabilities, and input validation issues dominate in both ARM and x86 binaries. Detailed dataset statistics and per-project function counts for both architectures are reported in Appendix A.

Moreover, it is worth noting that functions expressed in high-level code exhibit a median number of lines of code (LOC) that is less than half of their assembly counterparts, which is expected given the higher abstraction level of source code.

4.2 Decompilation Setup

For the neural decompilation phase, transformer-based models were fine-tuned to translate assembly code into C/C++ source code. To assess the suitability of different neural architectures for translation, we adopted several representative models from diverse design paradigms. All models are pre-trained on large-scale code corpora and are subsequently fine-tuned on VulDec for the neural decompilation task.

- *Qwen2.5-Coder 3B* (Hui et al. 2024), a decoder-only large language model optimized specifically for code, included to evaluate the capabilities of a highly capable, dense, instruction-tuned model trained on extensive multilingual code repositories.
- *DeepSeek-Coder-Instruct 1.3B* (Guo et al. 2024) is selected to assess the performance of a instruction-tuned decoder-only model that leverages repository-level code completion and a mix of project-level context during its pre-training phase.
- *CodeT5+ 220M* (Wang et al. 2023) is employed as a reference encoder–decoder model, leveraging span-denoising pre-training and a hybrid architecture that has demonstrated strong performance in both code understanding and generation tasks.
- *CodeBERT 125M* (Feng et al. 2020), an encoder-only model based on RoBERTa, is included to evaluate the effectiveness of rich contextual representations during the encoding of assembly instructions. In our approach, it is coupled with a seq-to-seq decoder, forming a full encoder–decoder architecture.

- *CodeGPT 124M* (Radford et al. 2019) and *CodeParrot 110M* (Tunstall et al. 2022), both decoder-only autoregressive models, are adopted to assess the flexibility of generation-only architectures when applied to a translation task not explicitly targeted during pre-training.
- *PLBART 140M* (Ahmad et al. 2021), an encoder–decoder model derived from BART and trained using multi-task denoising objectives on code and natural language, is considered to explore its ability to reconstruct structured high-level code from low-level inputs.

We performed a stratified split between training, validation, and testing in a ratio of 90/5/5. The dataset was partitioned using a project-level stratified split, accounting for code length distribution. This rigorous separation avoids cross-project contamination and ensures that performance metrics are not skewed by variations in function size. Based on the previously discussed size discrepancy, we constrained the decompilation output to half the size of the input assembly representation. The maximum input sequence length (assembly) was fixed at 512 tokens, while the maximum output sequence length (C code) was set to 256 tokens, resulting in a dataset reduction of $\sim$ 50%. Samples exceeding the predefined token limits were discarded to prevent code truncation and to limit computational overhead. The selected thresholds ensure a trade-off between representational expressiveness and computational feasibility. To standardize training conditions across experiments, all neural models were optimized using common hyper-parameters. Model-specific hyper-parameters, such as the number of training epochs, batch size, beam size, and evaluation frequency, were adapted according to architectural complexity, memory constraints, and expected performance. Detailed training configurations for each architecture (x86 and ARM) are reported in the Appendix B.

4.3 Vulnerability Detection Setup

For the vulnerability detection phase, we adopted several models pretrained on large-scale code corpora. Specifically, we evaluated Qwen2.5-Coder 3B (Hui et al. 2024), DeepSeek-Coder-Instruct 1.3B (Guo et al. 2024), CodeT5 (Wang et al. 2021), CodeBERT (Feng et al. 2020), CodeGPT (Radford et al. 2019), CodeParrot (Tunstall et al. 2022), GraphCodeBERT Base (Guo et al. 2021), and PLBART (Ahmad et al. 2021). GraphCodeBERT extends the capabilities of CodeBERT by incorporating the inherent structure of source code. Unlike traditional models that treat code as a simple sequence of tokens, GraphCodeBERT leverages data flow graphs.

To ensure architectural consistency for the classification task, encoder-only models (e.g., CodeBERT, GraphCodeBERT) were directly fine-tuned with a classification head. For encoder–decoder models (e.g., CodeT5 and PLBART), we followed the approach established by Wang et al. (2021), retaining only the encoder component to extract sequence representations. For decoder-only models (e.g., CodeGPT and CodeParrot, based on the GPT architecture), we employed the standard sequence classification variant established by Radford et al. (2019), where prediction is performed using the hidden state of the last non-padding token.

All models are fine-tuned for source code vulnerability classification. Training data consists of a blend of VulDec and unused samples from DiverseVul (Chen et al. 2023) external to VulDec and the end-to-end evaluation. This integration aims to enrich data diversity while avoiding redundant instances.

To ensure consistency with the decompilation pipeline and enable a fair comparison between the source and decompiled code, input sequences were constrained to a maximum length of 256 tokens. Functions with fewer than 20 tokens or exceeding this limit were discarded. As a result, the dataset was reduced by $\sim$ 25%. This filtering step significantly reduced the number of vulnerable instances and further exacerbated the original class imbalance (3% vulnerability rate).

To address the severe class imbalance of the dataset, we introduced a validation phase aimed at identifying the *optimal imbalance mitigation strategy*. For each model, we benchmarked different mitigation techniques on the validation set, which was intentionally kept imbalanced (3% vulnerability prevalence) to mirror a realistic distribution. We evaluated the impact of standard training (*Raw*), cost-sensitive learning via *Class Weighting* (*CW*), and various configurations of *Under-Sampling* (*US*) aimed at altering the training distribution. The configuration that yielded the best trade-off between minority-class precision and recall on the validation set was then selected as the definitive training pipeline. The dataset is partitioned into training, validation, and test sets using a stratified 90/5/5 split to preserve class balance across all subsets. Further details on hyper-parameter tuning and training splits are provided in the Appendix B.

5 Experimental Evaluation

In this section, we evaluate the performance of our approach by analyzing its capabilities in the decompilation and vulnerability detection tasks.

5.1 Decompilation Task

The performance of LLMs was assessed using a set of complementary metrics, summarized in Table 2. We employed Edit Similarity (ES), BLEU-4, and ROUGE-L to quantify translation quality. The Edit Distance (ED), adopted by Hosseini and Dolan-Gavitt (2022) and Katz et al. (2018), measures structural fidelity by counting the minimal edits (insertions, deletions, and substitutions) needed to transform the decompiled output into the reference code. ES is a normalized version of ED in the range [0,1]. Given two token sequences A and B, its formula is

$$ES(A, B) = 1 - \frac{ED(A, B)}{\max(|A|, |B|)}, \tag{1}$$

where $|A|$ and $|B|$ are the lengths of the two sequences. The higher its value, the better the translation will be. BLEU-4 evaluates n-gram precision, and ROUGE-L captures structural and semantic overlaps. However, metrics such as BLEU, originally developed for natural text translation, may not be particularly appropriate for evaluating translation between code sequences (Liguori et al. 2023; Tran et al. 2019). Although

Table 2: Automatic evaluation of the neural decompilation task. Results are reported for x86 and ARM architectures

Model	# parameters	ISA	Edit Similarity	BLEU-4	ROUGE-L
Qwen2.5-Coder	3.0B	x86	**55.83**	40.50	**56.45**
		ARM	**56.60**	43.42	**55.96**
CodeT5+	220M	x86	55.55	**42.45**	37.34
		ARM	56.12	**44.41**	38.10
DeepSeek-Coder	1.3B	x86	49.18	29.76	45.97
		ARM	47.17	27.51	44.21
CodeBERT	125M	x86	47.39	26.27	35.16
		ARM	43.56	20.46	34.27
PLBART	140M	x86	34.29	14.34	25.19
		ARM	27.19	7.41	19.66
CodeGPT	124M	x86	23.96	2.60	20.78
		ARM	22.04	0.16	19.35
CodeParrot	110M	x86	27.14	3.95	24.11
		ARM	24.67	2.94	22.14

originally designed for natural language, these metrics provide useful insights into code translation, with ES being particularly suited for capturing fine-grained differences.

Across both x86 and ARM datasets, Qwen2.5-Coder (Hui et al. 2024) achieves the strongest overall performance, reaching an Edit Similarity of 55.83% on x86 and 56.60% on ARM, while dominating the token-level context coverage with a ROUGE-L score of 56.45% (x86) and 55.96% (ARM). Interestingly, *DeepSeek-Coder* (Guo et al. 2024), despite having fewer than half the parameters of Qwen, demonstrates highly competitive token-generation capabilities, scoring a remarkably high ROUGE-L of 45.97% on x86 and 44.21% on ARM. This underscores the substantial impact of scaling and advanced code-specific pre-training recipes.

A notable finding is the performance of CodeT5+ (Wang et al. 2023). Despite being an order of magnitude smaller than Qwen2.5-Coder, it achieves the highest overall BLEU-4 scores (42.45% on x86 and 44.41% on ARM) and highly competitive Edit Similarity values. This exceptional efficiency is highly attributable to its native encoder-decoder architecture and span-denoising pre-training objective. PLBART (Ahmad et al. 2021) ranks intermediate: its multi-task denoising improves robustness to corrupted inputs, yet it falls short of CodeT5 in sequence-level accuracy.

Small, pure autoregressive models, i.e., CodeGPT (Radford et al. 2019) and CodeParrot (Tunstall et al. 2022), fail drastically on this task. This failure underscores that generation-only architectures, when lacking sufficient capacity or bidirectional context during encoding, cannot effectively map low-level structural assembly sequences to high-level source code. Conversely, encoder-only models adapted to a seq-to-seq pipeline, i.e., CodeBERT (Feng et al. 2020), maintain much higher structural robustness, retaining an Edit Similarity above 43%, proving that robust contextual representation of assembly instructions is a critical prerequisite for meaningful translation.

The results further reveal a non-uniform influence of the target ISA on performance. For CodeT5 and Qwen, ARM binaries slightly outperform x86 across most metrics, suggesting a more regular or learnable instruction-to-source mapping. By contrast, DeepSeek, CodeBERT, and PLBART exhibit a more pronounced degradation on ARM, indicating greater sensitivity to architectural variation. Overall, however, no consistent trend clearly favors one ISA over the other, although the ARM architecture instruction set displays marginally lower syntactic complexity than x86, consistent with its RISC principles of fixed-length instructions, limited addressing modes, and a highly regular operand structure.

Table 3 reports the performance on the HumanEval benchmark (Chen et al. 2021) under `-O2` optimization, comparing our VulDec Decompiler against rule-based decompilation, Ghidra (National Security Agency 2019), and the specialized *LLM4Decompile* family (Tan et al. 2024).

Table 3: Human-Eval BenchMark (-opt 02) on VulDec x86 Decompiler

Framework	**Model**	**ES**	**Re-executability**
Ghidra	Base	6.19	15.24
Tan et al. (2024)	LLM4Decompile-End-6.7B	12.93	36.71
	Ghidra + LLM4Decompile-Ref-6.7B	13.42	47.56
	Ghidra + LLM4Decompile-Ref-22B	13.58	**59.76**
	Ghidra + LLM4Decompile-Ref-33B	13.63	43.90
VulDec Decompiler	Qwen2.5-Coder-3B	**32.82**	34.34

In terms of structural alignment, our framework yields a substantial improvement, reaching an edit similarity of 32.82%, which noticeably exceeds the scores of the other evaluated models. While hybrid multi-stage pipelines like *Ghidra + LLM4Decompile-Ref-22B* achieve higher re-executability (59.76%), VulDec achieves a competitive 34.34%, performing closely to the standalone *LLM4Decompile-End-6.7B* (36.71%) despite utilizing a significantly smaller model (3B parameters) and operating without external decompilation outputs. As expected, the baseline Ghidra exhibits a low Edit Similarity of 6.19%. This occurs because rule-based decompilers focus on functional reconstruction rather than matching human coding styles, often generating nested structures, unpromoted types, and generic variable names that fail to compile directly or match the reference source text.

5.2 Vulnerability Detection Task

Table 4 summarizes the validation results for the encoder-only *CodeBERT* model to identify the optimal class-imbalance mitigation strategy. To ensure a valid assessment, all configurations are evaluated on a validation set that preserves the operational distribution (3% vulnerability prevalence). We benchmarked several *under-sampling* (US) configurations, systematically varying the majority-to-minority class ratios, namely

50:50, 70:30, and 80:20. In tandem, we evaluated a cost-sensitive *class weighting* (CW) scheme designed to penalize minority-class misclassifications. Specifically, the weight assigned to each class was defined as inversely proportional to its training prevalence; for instance, under the 80:20 US configuration, the resulting class weights are set to 1:4 for the majority and minority classes, respectively.

Table 4: Class-imbalance mitigation strategies using CodeBERT on validation set (3% vulnerability prevalence). CW: Class Weighting; US: Under-Sampling

Strategy	**Vul-Prec**	**Vul-Recall**	**Vul-F1**	**F1-score (macro avg)**
Raw	0.463	0.097	0.160	0.571
Raw + CW	**0.532**	0.115	0.189	0.586
US 50-50	0.070	0.707	0.128	0.466
US 70-30	0.127	0.427	0.196	0.567
US 80-20	0.167	0.336	0.223	0.591
US 80-20 + CW	0.171	**0.415**	**0.243**	**0.599**

The empirical results highlight a sharp trade-off between precision and recall. Standard training configurations (*Raw* and *Raw + CW*) under-represent the minority class, leading to negligible recall. Conversely, a perfectly balanced training distribution, US 50-50, over-compensates, causing an unacceptable collapse in precision (0.070) and a severe degradation of the macro F1-score. The data indicate that shifting towards moderate under-sampling ratios stabilizes the model. For *CodeBERT*, the best performance across all integrated metrics is achieved by the hybrid **US 80-20 + CW** strategy, which yields the top *Vul-Recall* (0.415) and *Macro F1-score* (0.599). We extended this validation setup to one model per architecture family in Appendix C. The US 80-20 + CW configuration consistently proved to be the optimal choice for the majority of architectures. However, for *Qwen2.5-Coder* and *DeepSeek-Coder* architectures, the balanced **US 50-50** ratio yielded the highest overall performance, suggesting a higher tolerance for class balance adjustments. Consequently, each model was trained using its respective optimal configuration in the subsequent evaluation phases.

Table 5 reports model performance on *project-level in-distribution* (ID) data, i.e., code from projects already seen during training for the vulnerability detection task. To ensure a realistic operational evaluation, all architectures are tested on an imbalanced test set with a 3% vulnerability prevalence. Models are evaluated using standard metrics (Accuracy, Precision, Recall, Macro F1) alongside minority-class specific metrics (*Vul-Precision*, *Vul-Recall*, and *Vul-F1*) to assess their behavior on the heavily imbalanced minority. The results show that overall *Accuracy* is an uninformative metric in this highly imbalanced scenario; for instance, the baseline DiverseVul (Chen et al. 2023) classifier, despite having been trained on an extended dataset incorporating additional external data sources, yields a high accuracy (0.923) but completely fails to effectively capture the minority class, resulting in an exceptionally low macro F1-score (0.472). The rule-based static analyzer, *FlawFinder* (Wheeler 2021), exhibits poor capabilities in identifying vulnerable samples, yielding a critical *Vul-F1* of 0.077.

Table 5: Vulnerability prediction performance on an imbalanced in-distribution test set (3% vulnerable samples). Reported metrics refer to the vulnerable class and overall macro performance

In-distribution Test Set							
Model	**Vul-P**	**Vul-R**	**Vul-F1**	**Acc**	**Prec**	**Recall**	**F1**
DiverseVul*	–	–	–	0.923	0.518	0.433	0.472
FlawFinder	0.055	0.130	0.077	0.898	0.512	0.527	0.511
CodeT5 (base)	0.150	0.309	0.202	0.920	0.563	0.625	0.580
CodeParrot	0.164	0.276	0.206	0.930	0.569	0.614	0.584
DeepSeek-Coder	0.180	0.260	0.213	**0.937**	0.577	0.610	0.590
Qwen2.5-Coder	0.180	0.301	0.226	0.932	0.579	0.627	0.596
CodeGPT	0.160	0.417	0.231	0.909	0.569	0.671	0.591
GraphCodeBERT	0.168	0.423	0.240	0.912	0.574	0.676	0.597
CodeBERT	0.176	0.398	0.244	0.919	0.577	0.667	0.601
PLBART	**0.180**	**0.444**	**0.257**	0.916	**0.580**	**0.688**	**0.606**

* Trained on an extended dataset incorporating additional external sources

Overall, the absolute performance metrics across all evaluated neural architectures remain relatively modest, with the highest vulnerability-specific F1-score (*Vul-F1*) reaching only 0.257. This concern highlights the intrinsic complexity and persistent challenges of software vulnerability detection under severe real-world class imbalance, an open problem widely documented in the literature (Ahmed et al. 2026; Chakraborty et al. 2022; Sheng et al. 2024).

Among the evaluated neural architectures, PLBART (Ahmad et al. 2021) achieves the best results for the security task, maximizing both *Vul-Recall* (0.444) and *Vul-F1* (0.257). Encoder-based models (CodeBERT and GraphCodeBERT) track closely, maintaining a balanced trade-off with a *Vul-F1* of approximately 0.24. Conversely, decoder-only architectures, i.e., CodeParrot and CodeGPT, along with standard *seq2seq* baselines CodeT5, experience a visible performance degradation on the minority class, showing lower precision and restricted recall profiles. A particularly notable trend emerges when examining larger model architectures (> 1B parameters). Despite their superior parameter scale, these models fail to outperform significantly smaller encoder and sequence-to-sequence networks. *DeepSeek-Coder* and *Qwen2.5-Coder* report *Vul-F1* scores of 0.213 and 0.226, respectively, placing them in the lower range of learning-based models. Our observation that larger models do not strictly yield superior performance mirrors empirical findings in the literature (Chen et al. 2023), where scaling model parameters similarly produced non-monotonic improvements in vulnerability detection.

5.3 End-to-End Evaluation

Once the core pipeline components, specifically *Qwen2.5-Coder* (Hui et al. 2024) for decompilation and *PLBART* (Ahmad et al. 2021) for vulnerability detection, have been established, the proposed framework undergoes an end-to-end inference evaluation.

The evaluation is conducted on functions extracted from projects that are disjoint from the training set, *i.e., project-level out-of-distribution* (OOD), in order to ensure rigorous generalization. This test set comprises over a thousand real-world functions gathered from the following open-source projects: `PHP`, `openjpeg` and `poppler`. Furthermore, this evaluation set preserves the realistic operational distribution with a 3% vulnerability prevalence, effectively exposing the framework to novel variations in coding style, API usage, and program structures. For all subsequent experiments, we focus exclusively on binaries compiled for the x86 architecture.

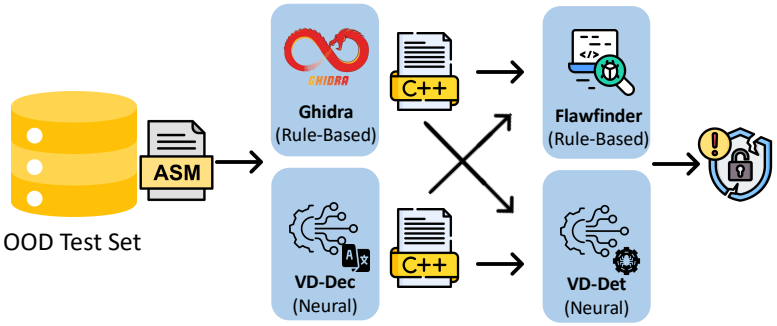


Fig. 5: Architectural overview of the end-to-end evaluation via cross-combination pipelines evaluating rule-based and neural stages for decompilation and detection

We integrate two industry-standard, rule-based security and reverse-engineering tools into our evaluation. For the translation stage, we employ *Ghidra* (National Security Agency 2019)[2], a widely adopted software reverse engineering framework driven by deterministic, rule-based decompilation heuristics. For the vulnerability detection stage, we adopt *Flawfinder* (Wheeler 2021), a well-established static analysis tool that scans source code for security flaws by matching rigid syntactic patterns.

To assess the joint impact of decompilation quality on downstream security tasks, we perform an end-to-end evaluation testing four distinct pipeline configurations. As illustrated in Fig. 5, the process orchestrates a translation stage (mapping binary assembly to source code) followed by a vulnerability detection stage. Specifically, our experiments contrast the fully traditional pipeline (*Ghidra* $\rightarrow$ *Flawfinder*) against our fully neural framework VulDec, alongside two cross-combined hybrid configurations (*Ghidra* $\rightarrow$ VD-Det and VD-Dec $\rightarrow$ *Flawfinder*). This cross-evaluation allows us to isolate the individual contributions of both the decompilation and the detection components under realistic conditions.

Table 6 reports the performance across both translation metrics (Edit Similarity, BLEU-4) and classification metrics. The end-to-end evaluation yields several critical insights outlined below.

[2]Since Ghidra cannot process isolated assembly code without a complete binary structure, we built a preprocessing pipeline that leverages *Keystone* (Quynh et al. 2024) to turn target function fragments back into raw executable binaries, allowing Ghidra's headless mode to decompile them.

Table 6: End-to-end evaluation on an imbalanced out-of-distribution test set (3% vulnerability prevalence). The columns report cross-combinations of rule-based and neural tools for both decompilation and vulnerability detection

Out-of-distribution Test Set						
Pipeline	**ES**	**BLEU-4**	**Vul-P**	**Vul-R**	**Vul-F1**	**F1**
Ghidra → Flawfinder	26.27	9.19	0.00	0.00	0.00	0.49
Ghidra → VD-Det	26.27	9.19	0.03	0.04	0.03	0.50
VD-Dec → Flawfinder	**36.83**	**19.36**	0.04	0.07	0.05	0.51
VD-Dec → VD-Det	**36.83**	**19.36**	**0.14**	**0.28**	**0.19**	**0.57**

End-to-End Pipeline Analysis and Cross-Evaluation. The traditional rule-based pipeline, i.e., *Ghidra → Flawfinder*, fails entirely in this realistic setup and collapses on the minority class, reporting a Vul-F1 of 0.00. This highlights the inability of rigid syntactic pattern-matchers to generalize over the structural variations generated during standard decompilation. This architectural limitation is slightly resolved by our neural decompiler, which establishes a superior baseline over Ghidra. It shows a strong regularizing effect on downstream tools. Replacing Ghidra with VD-Dec allows even a traditional analyzer, i.e., Flawfinder, to capture vulnerabilities (Vul-F1 = 0.05). The fully neural pipeline, *VD-Dec → VD-Det*, achieves the best performance, maximizing Vul-Recall (0.28) and tracking the highest global F1-score.

The Generalization Gap: ID vs. OOD Robustness. A comparative analysis between the evaluation on the in-distribution test set and the out-of-distribution set reveals an *evident performance degradation*, particularly affecting the minority-class recall, which drops from 0.44 to 0.28 for the optimal configuration.

This behavior exemplifies a classic *out-of-distribution generalization problem*, a well-documented limitation in the current state-of-the-art. Recent benchmarks, such as Li et al. (2025), prove that high in-distribution performance is often a misleading metric driven by spurious correlations, showing severe accuracy drops, e.g., from 0.70 to 0.49, when models are exposed to real-world OOD test sets. Similarly, Chakraborty et al. (2022) highlights that evaluating deep learning models on real-world test sets with different distributions triggers a disastrous average performance drop of $\sim -73\%$. The DiverseVul classifier (Chen et al. 2023) suffers a massive architectural collapse under a comparable domain shift, with its reported Macro F1-score dropping from 0.472 on the ID test set to a near-baseline 0.110 on the OOD test set. To further underline this vulnerability, frameworks like *SLODA* (Du et al. 2025) even exploit OOD features to intentionally mislead code models via adversarial attacks. More broadly, while an ID test set evaluates models on familiar software patterns encountered during training, an OOD set forces them to generalize over entirely novel coding conventions and library abstractions. This fundamental limitation leaves significant room for future optimization in the community.

Furthermore, the structural nature of the input code changes drastically between the two experimental setups: the ID evaluation operates directly on *source code*,

whereas this OOD end-to-end framework forces the detector to ingest *noisy, decompiled code*. Intuitively, the model would be expected to struggle with the syntactic artifacts and missing symbols introduced during translation. However, as shown in the following section, empirical results reveal a counterintuitive trend, proving that this noise is not the primary cause of the performance gap.

5.4 Impact of decompilation on vulnerability prediction

To further investigate the relationship between translation quality and security classification, Fig. 6 correlates the *Decompilation Fidelity (Edit Similarity)* with the downstream *Classification Accuracy* and *Macro F1-Score*.

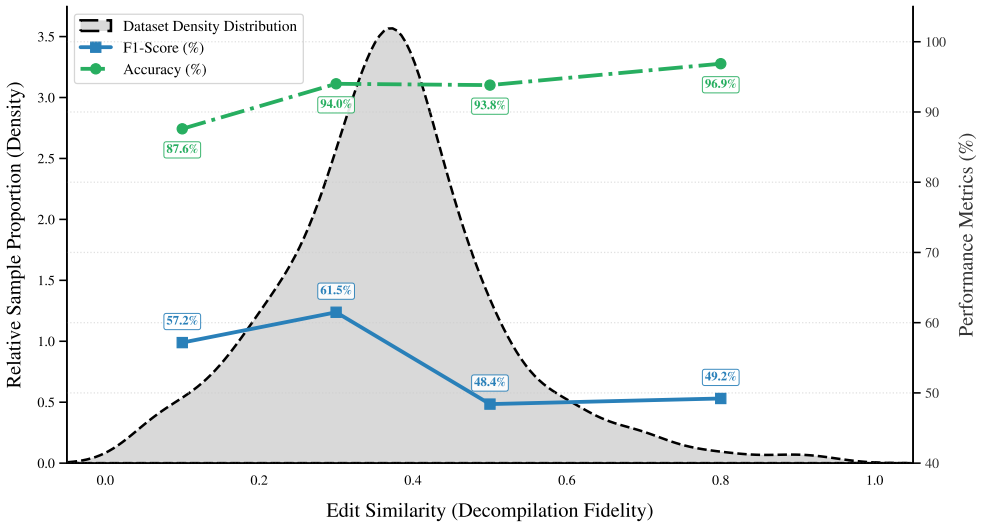


Fig. 6: Correlation analysis between Edit Similarity (Decompilation Fidelity) and downstream classification performance (Accuracy and F1-Score). The density curve (left) highlights the sample distribution across the test set, while the trend lines (right) illustrates the performance metrics values as structural translation quality increases

As illustrated in Figure 6, the dataset density distribution reveals that the vast majority of samples cluster around an Edit Similarity (ES) value of approximately 0.36. Visually, while downstream Accuracy follows a clear monotonic upward trend with increasing ES, the vulnerability detection F1-score exhibits a non-monotonic pattern, peaking at 61.5% near an ES of 0.3 before stabilizing around 49%.

Correlation Analysis. To rigorously validate these empirical observations, we performed a statistical correlation analysis examining the relationships between Edit Similarity, Accuracy, F1-score, and code scale using Spearman's rank (ρ) and Point-Biserial correlation tests at a 95% confidence level ($\alpha = 0.05$). The results reveal a *clear decoupling between syntactic fidelity and security semantics*. Specifically, while

Accuracy significantly correlates with structural quality, the F1-score shows no statistically meaningful dependency on Edit Similarity. This finding means that generating syntactically or structurally similar code does not inherently help the downstream model in capturing subtle security vulnerabilities. Furthermore, code scale acts as a penalty on generation performance: a larger function length (higher token count) significantly degrades both Edit Similarity and Accuracy, yet displays no statistically significant impact on the vulnerability detection F1-score.

Control Experiment. Finally, we conducted a control experiment by evaluating the detector directly on the *source code* of the OOD test set in order to rigorously isolate whether the observed OOD performance degradation stems from decompilation noise. Strikingly, the model yielded nearly *identical results*, with a negligible variance of ± 0.02 (maintaining a vulnerable-class recall of 0.28 and precision of 0.14). The control experiment alignment suggests that the decompilation process is not the primary driver of the performance drop, as the model faces the same difficulties even when analyzing clean source code. This result indicates that the detector has reached a generalization plateau; rather than capturing deep, universal security semantics, the LLM appears to rely on stylistic and syntactic details specific to the training distribution, leading to the same modest performance.

5.5 Effectiveness of VulDec Across Vulnerability Categories

The behavior of the vulnerability detection model was analyzed across vulnerability categories to assess potential category-level imbalances. The binary detection evaluation (safe vs. vulnerable) was extended to a multiclass setting using CWE annotations, allowing for the identification of both vulnerable code and specific weakness types. The VulDec dataset comprises 148 distinct CWE codes, some of which are missing. To reduce computational and interpretative complexity and to group similar vulnerability types, functions were consolidated into several macro-categories shown in Table 7. The assignment, reported in Appendix D, follows a custom taxonomy inspired by MITRE categories (The MITRE Corporation 2024b) and is designed to ensure that each function is annotated with its binary status, CWE identifier (when available), and the corresponding vulnerability group. Metrics are computed based on PLBART's evaluation of the in-distribution test set.

Table 7: Performance of vulnerability prediction across vulnerability groups reported in Appendix D and ID test set distribution

Group	Distr %	Prec	Rec	Acc	F1
Uncategorized	48.47	0.58	0.68	0.91	0.60
Memory Management and Concurrency	32.30	0.58	0.72	0.92	0.61
Validation and Authorization Issues	6.66	0.60	0.72	0.92	0.63
Taint Propagation Issues	7.89	0.59	0.67	0.94	0.62
Data Protection Issues	4.19	0.53	0.57	0.91	0.54
Insecure Configuration Issues	0.48	0.49	0.42	0.83	0.45

The distribution of functions across macro-categories reveals a marked imbalance, with *Uncategorized* samples and *Memory Management and Concurrency* dominating the dataset (48.47% and 32.30% prevalence, respectively). This layout aligns with traditional C/C++ repository distributions, where memory-related flaws represent the historical baseline. Conversely, categories like *Data Protection Issues* (4.19%) and *Insecure Configuration Issues* (0.48%) are severely underrepresented.

Performance, detailed in Table 7, varies across these families when evaluated through macro-averaged metrics. *Validation and Authorization Issues* achieved the highest localized detection capability, reaching a Macro Recall of 0.72 and a Macro F1-score of 0.63, closely followed by *Memory Management and Concurrency* (0.72 Recall, 0.61 F1-score). This suggests that the structural and contextual cues tied to buffer tracking and access control loops provide robust, recognizable patterns for the downstream detector.

In contrast, *Data Protection Issues* and *Insecure Configuration Issues* present the weakest macro performance. For the latter, the extreme scarcity of data (0.48% distribution) severely constrains the model's predictive alignment, dragging the Macro F1-score to 0.45 and the Macro Recall to 0.42.

While *Accuracy* remains consistently high (0.83 to 0.94), it is mathematically padded by the massive volume of non-vulnerable samples correctly identified by the model. The macro-averaged F1-score, settling around 0.60 for the main categories, provides a far more reliable proxy of the actual trade-off within each macro-group.

5.6 Assembly-Based Vulnerability Detection

To evaluate the impact of bypassing the decompilation stage, we trained a CodeBERT-based classifier to perform vulnerability detection directly on native x86 assembly using the VulDec dataset. This configuration, shown in Figure 7, establishes a direct assembly-to-vulnerability pipeline, effectively removing the intermediate abstraction layer. For training and subsequent evaluation, we adopted the same careful sample pre-processing procedure performed in Section 4.

Table 8: Assembly-based vulnerability detection on VulDec dataset

Class	Precision	Recall	F1-Score
Safe	0.89	1.00	0.92
Vul	0.00	0.00	0.00

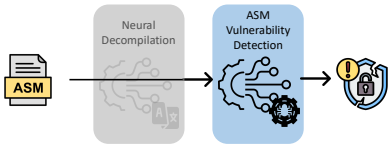


Fig. 7: Vulnerability detection on raw x86 assembly (no decompilation baseline)

The results, reported in Table 8, show strong performance in the Safe class (precision 0.89, recall 1.00, F1-score 0.92), while completely failing to identify vulnerable functions, resulting in zero precision, recall, and F1-score for the Vul class.

This behavior indicates that the model struggles to infer the higher-level semantic information required to recognize vulnerable behaviors. Assembly code lacks explicit control-flow abstractions, variable semantics, and contextual cues that are crucial for vulnerability reasoning. As a result, the model becomes overly biased toward Safe predictions, missing all vulnerable instances. These findings suggest that, despite simplifying the analysis pipeline, decompilation remains a necessary step for enabling effective vulnerability detection within VulDec.

6 Lessons Learned

(i) *Applicability of neural decompilation to cybersecurity.* The findings of this work suggest that the adoption of neural decompilation techniques for cybersecurity-related tasks constitutes a promising avenue for future research, potentially enabling new applications such as binary malware analysis. Although the decompiled C/C++ code exhibits limited syntactic fidelity, the preservation of its semantic content allows it to be effectively leveraged by learning-based approaches, which predominantly base their predictions on recurring patterns and structural features.

(ii) *Effectiveness of VulDec Dataset.* The experimental results indicate that the VulDec dataset shows promise as a viable benchmark for training both decompilation and vulnerability detection models. The evaluation on OOD test set yields encouraging outcomes, indicating the effectiveness of the proposed methodology. Nevertheless, the performance degradation observed during the training phase highlights the limitations imposed by the adopted computational resources and model capacity, suggesting that the use of more advanced architectures and greater computational power could further enhance the framework's effectiveness. Furthermore, the joint availability of low-level representations, high-level source code, and vulnerability annotations provided by the dataset opens up several promising research directions that lie beyond the scope of the contributions presented in this work.

(iii) *VulDec Framework Deployment Context.* Our empirical results indicate that while the framework achieves discrete performance, its practical utility is best realized as a *first-pass screening tool* to support developers during preliminary code reviews, in line with observations by Do et al. (2022) and Yedida et al. (2023). Rather than acting as an autonomous triage system, the tool is intended to highlight potential high-risk regions to guide manual inspection, thereby reducing the search space for human experts. This lightweight decision-support role aligns well with the requirements of assistant tools in software maintenance and reverse engineering.

7 Related Work

This work builds upon our previous exploratory study by Cotroneo et al. (2025), in which we introduced a preliminary pipeline for binary vulnerability detection based on neural decompilation. While this study demonstrated the feasibility of the approach, it relied on synthetically generated data, specifically the Juliet dataset (NIST 2017), thus limiting its applicability to real-world software. In contrast, the present work addresses this limitation by introducing VulDec, an ad-hoc dataset constructed from

real-world projects, and by extending the proposed pipeline to support multiple architectures, including both x86 and ARM, whereas the previous study was restricted to x86 binaries.

Neural decompilation is an innovative research trend in binary analysis. Recent work has explored the use of learning-based approaches for binary decompilation. Hosseini and Dolan-Gavitt (2022) introduced *Beyond The C*, a neural decompiler based on the fairseq transformer. This solution is retargetable for several high-level programming languages, such as C, OCaml, Go, and Fortran. The results of their study are promising for all languages, scoring the best result in terms of ED (54%) on the decompilation of C/C++ code (training on $2,000,000$ samples). Katz et al. (2018) proposed an approach for neural decompilation using Recurrent Neural Networks. The solution was trained and evaluated on snippets of binary machine code compiled from C source code. This solution achieved an ED equal to 30%, training on about $700,000$ samples. Tan et al. (2024) introduced a family of large-scale LLMs called *LLM4Decompile*, trained to translate binary code into high-level source code, achieving substantial improvements in readability and re-executability over traditional tools such as Ghidra (National Security Agency 2019) and general-purpose models.

While the previously mentioned works (Hosseini and Dolan-Gavitt 2022; Katz et al. 2018; Tan et al. 2024) focus on improving the quality of decompiled code as an end goal, our work differs by leveraging neural decompilation as an intermediate stage for downstream cybersecurity tasks, specifically black-box binary vulnerability detection, and by introducing a curated dataset that jointly aligns assembly, source-level semantics, and fine-grained vulnerability annotations. Existing approaches to binary vulnerability detection vary widely in terms of data sources, intermediate representations, learning paradigms, and evaluation methodologies, which makes direct experimental comparison across studies inherently challenging. Zheng et al. (2019) devised an approach to identify vulnerabilities in binary files using deep learning models, particularly Recurrent Neural Networks (RNN). To this end, this work leverages the Juliet C/C++ dataset (NIST 2017). Then, the binary code is lifted to LLVM (The LLVM Foundation 2024a) intermediate representation (IR), constituting the input for the RNNs. This solution can identify 4 CWEs with 90% accuracy. Schaad and Binder (2023) extended the previous approach (Zheng et al. 2019), leveraging several DL models, i.e., Recurrent Neural Networks (RNN), Long-Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU). Their solution can identify if a binary file is vulnerable with 88% accuracy, and classify 23 CWEs out of the 118 described in the Juliet dataset with 77% accuracy. Pewny et al. (2015) designed a cross-architecture solution to identify bug signatures in binary executables: the binaries are lifted to IR to unify the different assembly languages and then converted to control flow graphs (CFG) to compare basic blocks to the known signatures. This solution can identify 5 technology-specific vulnerabilities across three architectures (ARM, MIPS, x86), including the well-known *Heartbleed* vulnerability for OpenSSL (NIST 2014; Synopsys, Inc. 2014). *DiscovRE* (Eschweiler et al. 2016) is a prototype for identifying binary code bugs. It can identify two vulnerabilities, i.e., *Heartbleed* and *POODLE* for OpenSSL and SSLv3 respectively, and supports four architectures, namely x86, x64, ARM, and MIPS. DiscovRE measures structural similarities between the target code and the vulnerable

prototype through custom numeric features and metrics to identify the vulnerabilities. Dahl et al. (2020) developed a solution to identify stack-based buffer overflow vulnerabilities in binary files, along with a custom dataset for this task. First, the binary code is disassembled and then compressed to discard redundant information. Again, the vulnerability detection model is an LSTM, which achieves 90% accuracy in detecting stack-based buffer overflows. *VDISCOVER* (Grieco et al. 2016) is an ML-based tool for memory corruption detection. It comes with a custom dataset, *VDISCOVERY*, built from 1,039 programs from the Debian bug tracker. VDISCOVER can identify, with 55% accuracy, 4 types of memory corruptions: heap memory, stack memory, corruption of an argument in a function, and corruption of a pointer to a function.

Concurrently, recent advancements in source-level vulnerability detection have shifted toward multi-agent LLM reasoning frameworks. For instance, *VulTrial* (Widyasari et al. 2025) uses a courtroom-inspired multi-agent paradigm with role-specific agents to debate and explain source-level bugs. Similarly, *AEGIS* (Fang et al. 2026) executes multi-agent forensic verification over repository-level Code Property Graphs (CPG) to mitigate hallucinations. Crucially, the modular nature of our pipeline allows these sophisticated verification engines to be seamlessly integrated as alternative, high-fidelity downstream backends operating directly on our decompiled pseudo-C representation.

Most of the previously mentioned approaches to binary vulnerability detection rely on intermediate representations such as assembly-level tokens (Dahl et al. 2020), LLVM IR (Schaad and Binder 2023; Zheng et al. 2019), call graphs (Eschweiler et al. 2016), or combinations thereof (Pewny et al. 2015). To the best of our knowledge, none of these methods exploit decompiled C/C++ pseudo-code as the primary representation. This constitutes a key distinguishing aspect of our approach, as it not only enables vulnerability prediction but also provides an interpretable approximation of the original source code from which the binary was compiled. Furthermore, many of the strong classification results reported in the literature (Cotroneo et al. 2025; Schaad and Binder 2023; Zheng et al. 2019), particularly in multi-class settings, are largely attributable to the use of synthetic benchmarks. Our findings (Cotroneo et al. 2025), achieving up to 90% accuracy, corroborate this observation and highlight the impact of dataset realism on reported performance.

8 Threats To Validity

(i) *Dataset Construction and Integrity.* A key threat to validity arises from function matching between source code and assembly representations, which relies on symbol-name equality. To evaluate alignment fidelity in the absence of independent binary verification, we conducted a manual audit on 100 randomly sampled function pairs, confirming correct alignment in 95% of cases (95% Wilson CI [88.7%, 98.0%]). This suggests reasonable empirical accuracy for symbol-based alignment, indicating that incorrect function associations introduce minimal noise. Beyond alignment integrity, since our dataset builds upon DiverseVul (Chen et al. 2023), it directly inherits its documented $5-15\%$ label noise alongside its inherent class imbalance across vulnerability categories.

(ii) *Model selection and adaptation.* The validity of the results may also be affected by the choice of learning models (Hui et al. 2024; Guo et al. 2024; Feng et al. 2020; Guo et al. 2021; Radford et al. 2019; Tunstall et al. 2022; Wang et al. 2023) and their associated adaptation configurations. While smaller encoder-based and sequence-to-sequence baselines underwent full fine-tuning, computational resource constraints needed adapting larger generative LLMs, e.g., Qwen2.5-Coder and DeepSeek-Coder, via Low-Rank Adaptation (LoRA) over 3 epochs. While the selected models and fine-tuning regimes reflect widely adopted state-of-the-art approaches, this architectural and adaptation heterogeneity introduces a potential confounding factor, as more extensive hyperparameter optimization could influence model capacity.

(iii) *Metrics.* To evaluate the performance of neural decompilation, we selected output similarity metrics such as BLEU, ES, and ROUGE-L. This choice may threaten construct validity, as such metrics may not capture the correctness of the decompiled code. Nevertheless, output similarity metrics represent the most popular practice for evaluating neural machine translation, both for code-to-code (Hosseini and Dolan-Gavitt 2022; Katz et al. 2018), and NL-to-code translation tasks (Liguori et al. 2023).

(iv) *OOD Vulnerable Sample Size.* The limited absolute number of vulnerable samples (29 instances, reflecting a realistic $2.7\% \approx 3\%$ prevalence across $1,077$ functions) in our OOD test set introduces high statistical variance. A bootstrap analysis yields wide 95% confidence intervals for both Vul-Recall (95% CI $[0.12, 0.44]$) and Vul-F1 (95% CI $[0.07, 0.29]$). Consequently, while this setup preserves operational realism, individual prediction shifts under extreme imbalance significantly affect point estimates and should be considered when interpreting OOD degradation.

9 Conclusion

In this work, we introduced VulDec, a novel large-scale dataset that aligns C/C++ vulnerability annotations with their corresponding assembly representations, together with an end-to-end framework that leverages large language model–based neural decompilation for binary vulnerability detection. By translating binary executables into source-level pseudo-code and treating both representations as textual artifacts, our approach demonstrates how neural decompilation can serve as an effective abstraction for downstream cybersecurity tasks. Our primary objective was to advance research in black-box binary vulnerability detection by enabling realistic, data-driven studies and highlighting the potential of neural decompilation as a unifying bridge between low-level binaries and source-level semantics. The experimental evaluation of out-of-distribution, real-world binaries highlights the effectiveness and limitations of the proposed approach.

Future work will focus on enhancing the robustness of neural detectors against out-of-distribution samples, mitigating hallucinations in neural decompilers to ensure reliability in real-world scenarios, and incorporating semantic constraints and data-flow representations to enforce behavioral correctness in binary analysis.

Acknowledgments

This work was supported by the European Union – NextGenerationEU under the PNRR (Piano Nazionale di Ripresa e Resilienza), Mission 4, Component 2, Investment 'I.3.4 Borse PNRR transizioni digitali e ambientali', through Ministerial Decree no. 118 (02-03-2023). The authors would like to express their gratitude to the former student, *Marco Cimmino*, for his valuable contributions to the experimental evaluation of this work.

Appendix A Additional Details on VulDec Dataset

Table A1 reports the number of functions per project in the VulDec dataset for both ARM and x86 architectures, along with the corresponding project versions. The distribution of functions highlights the predominance of large-scale projects such as Linux, which contribute the majority of samples, while smaller projects provide fewer functions. All projects and their versions are identical across the two architectures, ensuring comparability between the ARM and x86 subsets. This selection of widely used open-source projects covers diverse application domains and software types, providing a representative and challenging benchmark for function-level vulnerability detection.

Table A1: Project versions and number of functions per architecture

Project	Version	ARM #functions	x86 #functions
linux	5.4.0	18,965	19,979
qemu	4.2.0	580	2,451
openssl	1.1.1f	1,303	1,311
vim	8.1.2269	857	843
mysql-server	8.0.40	147	145
samba	4.15.13	458	492
libxml2	2.9.10	1,000	1,001
FFmpeg	4.2.7	910	911
qpdf	9.1.1	92	92
wireshark	3.2.3	150	150
git	2.21.1	541	541
redis	5.0.7	171	171
curl	7.68.0	223	222
systemd	v245.4	670	669
libvirt	v6.0.0	172	172
bash	5.0	118	118
gnupg	2.2.19	367	365
mono	6.8.0.105	144	146
nautilus	3.36.3	326	326
OpenSC	0.20.0	460	462
libarchive	v3.4.0	275	275
krb5	1.17-final	308	308
flatpak	1.6.5	168	169
bluez	5.53	21	21

The dataset exhibits a skewed distribution dominated by large-scale systems, where the *linux* kernel alone contributes over 60% of the total functions. While this guaranties an exceptional representation of complex, enterprise-grade software patterns, it may limit the model's immediate generalizability to smaller projects, embedded systems, and IoT firmware. As a consequence, the learned features may fail to generalize effectively to the constrained coding conventions, lightweight libraries, and specific architectural optimizations typical of embedded and IoT environments.

Appendix B Experimental Setup Details

Table B2 summarizes the main statistics of the filtered and split x86 and ARM datasets used for neural decompilation, including the distribution across the training, development, and test sets. It also reports token-level statistics for both the input Assembly code and the output C code, showing that Assembly sequences are, on average, longer than their C counterparts. For completeness, the table additionally presents the statistics of the unbalanced dataset adopted for vulnerability detection.

Table B2: Unified statistics for neural decompilation (x86/ARM) and vulnerability detection preprocessed datasets

Category	**Feature**	**x86 → C**	**ARM → C**	**C → target**
Split	Total Examples	15,624	12,300	213,661
	Train	14,061	11,070	192,295*
	Dev	781	615	10,683
	Test	782	615	10,683
Assembly	Min Tokens	9	8	–
	Max Tokens	512	512	–
	Avg. Tokens	227.35	250.87	–
C Source	Min Tokens	10	11	20
	Max Tokens	256	256	256
	Avg. Tokens	100.26	95.24	99.45
Classes	Safe	–	–	204,664
	Vulnerable	–	–	8,997

* Train samples shown for the raw configuration; others (US) can be derived accordingly

Table B3 summarizes the key parameters used in the VulDec pipeline. The workflow follows a progressively decreasing input/output size: the first stage processes assembly code with a maximum length of 512 tokens and generates C/C++ code limited to 256 tokens, while the final output encodes binary vulnerability information (0/1). This design reflects the hierarchical nature of the task, moving from low-level instructions to high-level program semantics and finally to a compact, security-relevant representation. The dataset splits are chosen to balance sufficient training data with effective evaluation, using 90/5/5 for both tasks, ensuring model generalization and

Table B3: Training configuration for neural decompilation and vulnerability detection

Parameter	Neural Decompilation	Vulnerability Detection
Optimizer	AdamW	AdamW
Learning Rate	5×10^{-5}	2×10^{-5}
Epsilon (AdamW)	1×10^{-8}	1×10^{-8}
Weight Decay	0	0.01
Gradient Clipping	max_grad_norm = 1.0	–
Scheduler	1000 warm-up steps	Linear decay
Max Source Length	512 tokens	256 tokens
Max Target Length	256 tokens	Binary output (0/1)
Split	90/5/5	90/5/5

robust testing. A block of parameters, i.e., decision threshold, batch size, gradient accumulation, number of epochs, and beam size, was adapted for each model and each task to balance computational constraints, training time, and performance expectations, reflecting differences in model complexity and architecture. Training budgets were scaled by architecture and pre-training capacity. Encoder-decoder models (CodeT5, PLBART) required longer schedules (up to 10 epochs) for cross-attention alignment. Conversely, large decoder-only LLMs (Qwen, DeepSeek) were capped at 3 epochs to prevent overfitting. Smaller decoder-only models (CodeGPT, CodeParrot) were allocated an intermediate budget (5–8 epochs) to compensate for their lower capacity, while encoder-only models received minimal epochs due to their generative unsuitability.

Appendix C Class-Imbalance Strategy Optimization across Model Architectures

In order to adopt the optimal mitigation strategy under extreme class imbalance, we evaluated different under-sampling ratios on the validation set across representative model architectures, i.e., CodeBERT for Encoder-Only, PLBART for Encoder-Decoder, and CodeGPT for small Decoder-Only. Additionally, we explicitly performed this optimization for both modern decoder-only LLMs (Qwen2.5-Coder and DeepSeek-Coder) to rigorously verify their specific training dynamics.

As shown in Table C4, for standard encoder-based backbones as well as small decoder-only models, the combination of **US 80-20 + CW** consistently achieves the best balance between precision and recall, yielding the highest Vul-F1 and Macro F1 scores. Conversely, for modern decoder-only LLMs, the **US 50-50** sampling ratio proves to be the most effective strategy. Since class weights are defined as inversely proportional to the training class distribution, applying CW to a 50-50 balanced dataset results in a weight ratio of 1:1, making loss weighting mathematically redundant. Therefore, class balance was controlled entirely via dataset resampling without additional loss weighting.

Table C4: Class-imbalance mitigation strategies extended to all model architectures on validation set (3% vulnerability prevalence). CW: Class Weighting; US: Under-Sampling; SLM: Small Language Model; LLM: Large Language Model

Architecture (Model)	Strategy	Vul-Prec	Vul-Recall	Vul-F1	F1-score
Encoder Only SLM (CodeBERT)	US 50-50	0.070	0.707	0.128	0.466
	US 70-30	0.127	0.427	0.196	0.567
	US 80-20	0.167	0.336	0.223	0.581
	US 80-20 + CW	0.171	0.415	**0.243**	0.591
Encoder-Decoder SLM (PLBART)	US 50-50	0.072	0.725	0.131	0.469
	US 70-30	0.131	0.450	0.203	0.572
	US 80-20	0.170	0.360	0.231	0.587
	US 80-20 + CW	0.178	0.459	**0.257**	0.605
Decoder Only SLM (CodeGPT)	US 50-50	0.068	0.685	0.124	0.458
	US 70-30	0.120	0.420	0.187	0.559
	US 80-20	0.155	0.340	0.213	0.576
	US 80-20 + CW	0.162	0.420	**0.234**	0.586
Decoder Only LLM (Qwen2.5-Coder)	**US 50-50**	0.175	0.320	**0.226**	0.588
	US 70-30	0.182	0.225	0.201	0.570
	US 80-20	0.158	0.219	0.183	0.575
Decoder Only LLM (DeepSeek-Coder)	**US 50-50**	0.179	0.272	**0.221**	0.585
	US 70-30	0.162	0.205	0.181	0.571
	US 80-20	0.140	0.165	0.151	0.560

Appendix D CWE Macrocategory Mapping

Table D5 presents the costum mapping between the defined macro-categories and the corresponding CWE identifiers. Each macro-category groups vulnerabilities sharing common security characteristics

Table D5: Mapping between macrocategories and associated CWE identifiers

Macro-Category	Included CWEs
Taint Propagation Issues	CWE-20, CWE-74, CWE-77, CWE-79, CWE-89, CWE-116
Insecure Configuration Issues	CWE-16, CWE-611, CWE-276, CWE-732, CWE-933
Data Protection Issues	CWE-200, CWE-201, CWE-312, CWE-319, CWE-522
Validation and Authorization Issues	CWE-703, CWE-285, CWE-862, CWE-863, CWE-306, CWE-807
Memory Management and Concurrency	CWE-119, CWE-120, CWE-125, CWE-416, CWE-476, CWE-362, CWE-401
Uncategorized (Remaining CWEs)	This category aggregates all safe and vulnerable functions for which the CWE column is empty.

Appendix E Decompiled Code: Qualitative Analysis

The qualitative analysis reveals both strengths and limitations of the x86 neural decompiler. As shown in Table E6, we report four pairs of reference and decompiled functions extracted from the test set.

Tier 1: Perfect Structural Match and Contextual Renaming. This tier represents the ideal success scenario for our neural framework. As shown in the `store_tabletJitterDelay` routine from the Linux Kernel input subsystem, the generated control-flow graph and syntactic blocks mirror the ground truth perfectly. The marginal reduction in the ES score (95.21%) is exclusively caused by a variable renaming effect, where the original single-character identifier `j` is translated into `delay`. Far from being a flaw, this behavior highlights the model's ability to extract semantic context from surrounding tokens (e.g., the function name and the structure field `jitterDelay`).

Tier 2: Syntactic Shift with Functional Equivalence. The second tier highlights cases where the model demonstrates a deep semantic understanding derived from its pre-training phase. In the `i2c_smbus_read_byte_data` function, the ground truth implements a compact inline instruction: `return (status < 0) ? status : data.byte;`. VulDec decompiler successfully interprets the low-level conditional branch layout but decomposes it into an explicit, structurally expanded code path: `if (status < 0) return status; return data.byte;`. Although this syntactic variance induces a minor drop in token-level similarity (93.15%), the decompiled function remains 100% functionally equivalent.

Tier 3: Semantic and Type Shift. The intermediate tier (ES: 58.15%) uncovers critical insights regarding prompt-induced biases in generative models. In the `cmp_extents_reverse` example, the original code performs a standard three-way ascending comparison. However, the presence of the word `"reverse"` within the function identifier biases the autoregressive decoding, prompting the LLM to assume a descending sort order. Consequently, the model generates a specular return logic, swapping the `1` and `-1` outputs. Additionally, it experiences a type misalignment, injecting an extraneous pointer indirection (`*(const struct extent_map **)a`) and swapping the `uid_gid_extent` type with `extent_map`. Despite the semantic inversion, the underlying conditional structure (`if-if-return`) is accurately preserved.

Tier 4: Catastrophic Hallucination and Loop. This final tier (ES: 17.62%) embodies the catastrophic failure modes typical of autoregressive language models when handling unaligned binary syntax or hard code structures. Our extensive empirical evaluations indicate that severe hallucinations generally manifest as one of four cases:

1. premature decoding termination immediately after generating the function signature, omitting the function body entirely;
2. generation of a correct code block followed by arbitrary syntactic noise, unaligned code paths, or the same block repeated in a loop;
3. the fluent generation of a structurally correct but completely unrelated function;

Table E6: Qualitative analysis of VulDec neural decompilation outputs across different performance tiers sorted by Edit Similarity (ES)

Ground Truth	VulDec Decompiled

store_tabletJitterDelay(struct device *dev,
  struct device_attribute *attr, const char *buf,
  size_t count)
{
  struct aiptek *aiptek = dev_get_drvdata(dev);
  int err, j;
  err = kstrtoint(buf, 10, &j);
  if (err) return err;
  aiptek->newSetting.jitterDelay = j;
  return count;
}
	store_tabletJitterDelay(struct device *dev,
	  struct device_attribute *attr, const char *buf,
	  size_t count)
	{
	  struct aiptek *aiptek = dev_get_drvdata(dev);
	  int err, delay;
	  err = kstrtoint(buf, 10, &delay);
	  if (err) return err;
	  aiptek->newSetting.jitterDelay = delay;
	  return count;
	}

Edit Similarity (ES): 95.21%	

s32 i2c_smbus_read_byte_data(const struct i2c_client
  *client, u8 command)
{
  union i2c_smbus_data data;
  int status;
  status = i2c_smbus_xfer(client->adapter, client->addr, client->flags, I2C_SMBUS_READ, command, I2C_SMBUS_BYTE_DATA, &data);
  return (status < 0) ? status : data.byte;
 	
}
	s8 i2c_smbus_read_byte_data(const struct i2c_client
	  *client, u8 command)
	{
	  union i2c_smbus_data data;
	  int status;
	  status = i2c_smbus_xfer(client->adapter, client->addr, client->flags, I2C_SMBUS_READ, command, I2C_SMBUS_BYTE_DATA, &data);
	  if (status < 0) return status;
	  return data.byte;
	}

Edit Similarity (ES): 93.15%	

static int cmp_extents_reverse(const void *a, const void *b) {
  const struct uid_gid_extent *e1 = a;
 	
  const struct uid_gid_extent *e2 = b;
 	
  if (e1->lower_first < e2->lower_first)
  return -1;
  if (e1->lower_first > e2->lower_first)
  return 1;
  return 0;
}
	static int cmp_extents_reverse(const void *a, const void *b) {
	  const struct extent_map *em_a = *(const struct extent_map **)a;
	  const struct extent_map *em_b = *(const struct extent_map **)b;
	  if (em_a->start < em_b->start)
	  return 1;
	  if (em_a->start > em_b->start)
	  return -1;
	  return 0;
	}

Edit Similarity (ES): 58.15%	

xmlRegStatePtr xmlRegNewState(xmlRegParserCtxtPtr ctxt) {
  xmlRegStatePtr ret;
  ret = (xmlRegStatePtr) xmlMalloc(sizeof(xmlRegState));
  if (ret == NULL) {
  xmlRegexpErrMemory(ctxt, "allocating state");
  return(NULL);
  }
  memset(ret, 0, sizeof(xmlRegState));
  ret->type = XML_REGEXP_TRANS_STATE;
  ret->mark = XML_REGEXP_MARK_NORMAL;
 	
  return(ret);
}
	xmlRegStatePtr xmlRegNewState(xmlRegParserPtr parser) {
	  xmlRegParserStatePtr state;
	  state = (xmlRegParserStatePtr) xmlMalloc(sizeof(xmlRegParserState));
	  if (state == NULL) return(NULL);
	
	  state->state = XML_REG_STATE_INITIAL;
	
	  state->error = 0;
	  state->errorLine = 0;
	  state->errorColumn = 0;
	  state->errorOffset = 0;
	  ...
	  /* ... truncated due to infinite repetition loop ... */
	}

Edit Similarity (ES): 17.62%	

4. a severe collapse of the model's self-attention stopping criteria, leading to an infinite repetition loop on a single line of code.

The `xmlRegNewState` routine from `libxml2` showcases this latter attention collapse pattern. After hallucinating a non-existent error-tracking structure (`errorLine`, `errorColumn`, etc.), the decoder becomes trapped in an infinite loop, continuously reiterating identical allocation sequences until truncated by the maximum token generation boundary, leaving an incomplete and non-compilable artifact.

References

Ahmad, W.U., Chakraborty, S., Ray, B., Chang, K.-W.: Unified Pre-training for Program Understanding and Generation (2021). https://arxiv.org/abs/2103.06333

Ahmed, M.B.U., Harzevili, N.S., Shin, J., Pham, H.V., Wang, S.: Secvuleval: Context-aware benchmarking of llms for vulnerability detection. In: Proceedings of the 3rd ACM International Conference on AI-Powered Software. AIware '26, pp. 388–396. Association for Computing Machinery, New York, NY, USA (2026). https://doi.org/10.1145/3805760.3814932 . https://doi.org/10.1145/3805760.3814932

Ayewah, N., Pugh, W., Morgenthaler, J.D., Penix, J., Zhou, Y.: Evaluating static analysis defect warnings on production software. In: Proceedings of the 7th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering, pp. 1–8 (2007)

Avast Software: RetDec: Retargetable Machine-Code Decompiler. Available at https://github.com/avast/retdec. Accessed: 2026-03-03 (2017). https://github.com/avast/retdec

Chen, Y., Ding, Z., Alowain, L., Chen, X., Wagner, D.: DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection (2023)

Cotroneo, D., De Luca, R., Liguori, P.: Devaic: A tool for security assessment of ai-generated code. Information and Software Technology **177**, 107572 (2025) https://doi.org/10.1016/j.infsof.2024.107572

Cotroneo, D., Grasso, F.C., Natella, R., Orbinato, V.: Can neural decompilation assist vulnerability prediction on binary code? In: Proceedings of the 18th European Workshop on Systems Security. EuroSec'25, pp. 26–32. Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3722041.3723097 . https://doi.org/10.1145/3722041.3723097

Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep Learning Based Vulnerability Detection: Are We There Yet? (2022). https://doi.org/10.1109/TSE.2021.3087402

Chen, M., Tworek, J., Jun, H., Yuan, Q., Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov,

M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W.: Evaluating Large Language Models Trained on Code (2021). https://arxiv.org/abs/2107.03374

Cao, Y., Zhang, R., Liang, R., Chen, K.: Evaluating the effectiveness of decompilers. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2024, pp. 491–502. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3650212.3652144 . https://doi.org/10.1145/3650212.3652144

Dahl, W.A., Erdodi, L., Zennaro, F.M.: Stack-based buffer overflow detection using recurrent neural networks. arXiv preprint arXiv:2012.15116 (2020)

Silva, A.F., Kind, B.C., Souza Magalhães, J.W., Rocha, J.N., Ferreira Guimarães, B.C., Quinão Pereira, F.M.: Anghabench: A suite with one million compilable c benchmarks for code-size reduction. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 378–390 (2021). https://doi.org/10.1109/CGO51591.2021.9370322

Do, L.N.Q., Wright, J.R., Ali, K.: Why do software developers use static analysis tools? a user-centered study of developer needs and motivations. IEEE Transactions on Software Engineering **48**(3), 835–847 (2022) https://doi.org/10.1109/TSE.2020.3004525

Du, X., Wen, M., Wang, H., Wei, Z., Jin, H.: Statement-level adversarial attack on vulnerability detection models via out-of-distribution features **2**(FSE) (2025) https://doi.org/10.1145/3729403

Eschweiler, S., Yakdan, K., Gerhards-Padilla, E., *et al.*: discovre: Efficient cross-architecture identification of bugs in binary code. In: Ndss (2016)

Fang, S., Ding, W., Cao, Z., Yang, Z., Xu, B.: AEGIS: From Clues to Verdicts – Graph-Guided Deep Vulnerability Reasoning via Dialectics and Meta-Auditing (2026). https://arxiv.org/abs/2603.20637

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A Pre-Trained Model for Programming and Natural Languages (2020)

Fan, J., Li, Y., Wang, S., Nguyen, T.N.: A c/c++ code vulnerability dataset with code changes and cve summaries. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3379597.3387501 .

https://doi.org/10.1145/3379597.3387501

Grieco, G., Grinblat, G.L., Uzal, L., Rawat, S., Feist, J., Mounier, L.: Toward large-scale vulnerability discovery using machine learning. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy (2016)

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M.: GraphCodeBERT: Pre-training Code Representations with Data Flow (2021). https://arxiv.org/abs/2009.08366

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., Liang, W.: DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence (2024). https://arxiv.org/abs/2401.14196

Hosseini, I., Dolan-Gavitt, B.: Beyond the c: Retargetable decompilation using neural machine translation. arXiv preprint arXiv:2212.08950 (2022)

Hex-Rays: IDA Pro. Hex-Rays SA, Liège, Belgium (2025). Hex-Rays SA. Versione 8.x. https://hex-rays.com/ida-pro/

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., Lin, J.: Qwen2.5-Coder Technical Report (2024). https://arxiv.org/abs/2409.12186

Jain, R., Gervasoni, N., Ndhlovu, M., Rawat, S.: A code centric evaluation of c/c++ vulnerability datasets for deep learning based vulnerability detection techniques. In: Proceedings of the 16th Innovations in Software Engineering Conference. ISEC '23. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3578527.3578530 . https://doi.org/10.1145/3578527.3578530

Katz, D.S., Ruchti, J., Schulte, E.: Using recurrent neural networks for decompilation. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER) (2018). IEEE

Li, Y., Bui, N.T., Zhang, T., Weyssow, M., Yang, C., Zhou, X., Jiang, J., Chen, J., Huang, H., Nguyen, H.H., Ho, C.Y., Tan, J., Li, R., Yin, Y., Ang, H.W., Liauw, F., Ouh, E.L., Shar, L.K., Lo, D.: Out of distribution, out of luck: How well can llms trained on vulnerability datasets detect top 25 cwe weaknesses? arXiv preprint arXiv:2507.21817 (2025)

Liguori, P., Improta, C., Natella, R., Cukic, B., Cotroneo, D.: Who evaluates the evaluators? on automatic metrics for assessing ai-based offensive code generators. Expert Systems with Applications (2023)

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681 (2018)

National Security Agency: Ghidra Software Reverse Engineering Framework. Fort Meade, MD (2019). Accessed: 2026-03-03. https://www.ghidra-sre.org/

NIST: CVE-2014-0160 Detail. National Vulnerability Database (NVD). Accessed: 2026-03-03 (2014). https://nvd.nist.gov/vuln/detail/cve-2014-0160

NIST: Juliet Test Suite C/C++ 1.3. Available at https://samate.nist.gov/SARD/test-suites/112. Accessed: 2026-03-03 (2017). https://samate.nist.gov/SARD/test-suites/112

Ni, C., Shen, L., Yang, X., Zhu, Y., Wang, S.: Megavul: A c/c++ vulnerability dataset with comprehensive code representations. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, pp. 738–742. ACM, ??? (2024). https://doi.org/10.1145/3643991.3644886 . http://dx.doi.org/10.1145/3643991.3644886

Pistoia, M., Chandra, S., Fink, S.J., Yahav, E.: A survey of static analysis methods for identifying security vulnerabilities in software systems. IBM Systems Journal **46**(2), 265–288 (2007) https://doi.org/10.1147/sj.462.0265

Pewny, J., Garmany, B., Gawlik, R., Rossow, C., Holz, T.: Cross-architecture bug search in binary executables. In: 2015 IEEE Symposium on Security and Privacy (2015). IEEE

Quynh, N.A., et al.: Keystone: A Lightweight Multi-Platform, Multi-Architecture Assembler Framework. GitHub. https://github.com/keystone-engine/keystone Accessed 2026-09-04

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M.: Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 757–762 (2018). https://doi.org/10.1109/ICMLA.2018.00120

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners. OpenAI blog **1**(8), 9 (2019)

Schaad, A., Binder, D.: Deep-learning-based vulnerability detection in binary executables. In: Foundations and Practice of Security: 15th International Symposium, FPS 2022, Ottawa, ON, Canada, December 12–14, 2022, Revised Selected Papers (2023). Springer

Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen,

J., Feng, S., Hauser, C., Kruegel, C., Vigna, G.: Sok: (state of) the art of war: Offensive techniques in binary analysis. In: 2016 IEEE Symposium on Security and Privacy (SP), pp. 138–157 (2016). https://doi.org/10.1109/SP.2016.17

Sheng, Z., Wu, F., Zuo, X., Li, C., Qiao, Y., Hang, L.: LProtector: An LLM-driven Vulnerability Detection System (2024). https://arxiv.org/abs/2411.06493

Synopsys, Inc.: The Heartbleed Bug. Originally published by Codenomicon. Accessed: 2026-03-03 (2014). https://heartbleed.com/

The LLVM Foundation: LLVM Compiler Infrastructure (2024). https://llvm.org/

The MITRE Corporation: Common Weakness Enumeration (CWE) (2024). https://cwe.mitre.org/

The Debian Project: Debian GNU/Linux – Packages. https://www.debian.org/distrib/packages. Accessed: 2025-01-15 (2025)

The LLVM Foundation: LLVM-objdump: LLVM object file dumper. Accessed: 2026-03-03 (2026). https://llvm.org/docs/CommandGuide/llvm-objdump.html

Tan, H., Luo, Q., Li, J., Zhang, Y.: Llm4decompile: Decompiling binary code with large language models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 3473–3487 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.203 . http://dx.doi.org/10.18653/v1/2024.emnlp-main.203

Tran, N., Tran, H., Nguyen, S., Nguyen, H., Nguyen, T.: Does bleu score work for code migration? In: 2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC) (2019). IEEE

Tunstall, L., Von Werra, L., Wolf, T.: Natural Language Processing with Transformers. " O'Reilly Media, Inc.", ??? (2022)

Wheeler, D.A.: Flawfinder: A static analysis tool for finding vulnerabilities in C/C++ source code. Available at https://dwheeler.com/flawfinder/. Accessed: 2026-03-03 (2021). https://dwheeler.com/flawfinder/

Wang, Y., Le, H., Gotmare, A.D., Bui, N.D.Q., Li, J., Hoi, S.C.H.: CodeT5+: Open Code Large Language Models for Code Understanding and Generation (2023)

Widyasari, R., Weyssow, M., Irsan, I.C., Ang, H.W., Liauw, F., Ouh, E.L., Shar, L.K., Kang, H.J., Lo, D.: Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents (2025). https://doi.org/10.1145/3744916.3773256 . https://arxiv.org/abs/2505.10961

Wang, Y., Wang, W., Joty, S., Hoi, S.C.H.: CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation (2021)

Wu, H., Wu, J., Wu, R., Sharma, A., Machiry, A., Bianchi, A.: Veribin: Adaptive verification of patches at the binary level. (2025). https://doi.org/10.14722/ndss.2025.230359

Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and understanding bugs in c compilers. In: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation. PLDI '11, pp. 283–294. Association for Computing Machinery, New York, NY, USA (2011). https://doi.org/10.1145/1993498.1993532 . https://doi.org/10.1145/1993498.1993532

Yedida, R., Kang, H.J., Tu, H., Yang, X., Lo, D., Menzies, T.: How to find actionable static analysis warnings: A case study with findbugs. IEEE Transactions on Software Engineering **49**(4), 2856–2872 (2023) https://doi.org/10.1109/TSE.2023.3234206

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks (2019)

Zheng, J., Pang, J., Zhang, X., Zhou, X., Li, M., Wang, J.: Recurrent neural network based binary code vulnerability detection. In: Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence (2019)